



Planificación de Sprint

Proyecto Educativo Gamificado — Synapxix

Proyecto: **Synapxix**

Fecha: **06-07-2026 (Semana 2)**

Metodología: **Ágil / Scrum**

Stack: **Nx · Angular · NestJS · Prisma · PostgreSQL**



Mensaje del Tech Lead:

Si la semana anterior quedó prolija, el sistema ya tiene autenticación funcionando de punta a punta. Esta semana el foco es la integridad de las sesiones de juego: actualmente el `userId` en las `game-sessions` está en `null`, lo cual hace que las recompensas y el progreso no se puedan ligar a un usuario real. Ese es el trabajo más crítico. Además, vamos a darle amor al flujo de los ejercicios de audio con validaciones robustas. ¡La calidad del dato es sagrada!



Asignaciones y Objetivos del Equipo



Felipe

5 DÍAS

URGENTE

Ligar `GameSession` al usuario autenticado (Backend)

OBJETIVO PRINCIPAL

Agregar autenticación a los endpoints de `game-session` y asegurar que el `userId` se tome del token JWT en lugar de quedar en `null`.

CONTEXTO TÉCNICO

En `server/src/app/game-session/game-session.controller.ts` hay un comentario explícito que dice: "`userId se mantendrá en null intencionalmente`". Esto es deuda técnica crítica. Hay que agregar `@UseGuards(JwtAuthGuard)`, extraer el `sub` del token con `@Request()` y pasarlo al servicio para que las sesiones queden ligadas al usuario real.

CRITERIOS DE ACEPTACIÓN (DEFINITION OF DONE)

- ☐ Los tres endpoints de `game-session` (`start`, `attempt`, `complete`) tienen `@UseGuards(JwtAuthGuard)`.
- ☐ El `userId` se extrae del token JWT y se persiste correctamente en la tabla `GameSession`.

- ☐ Las recompensas reclamadas post-sesión están ligadas al usuario correcto.
- ☐ Se elimina el comentario de deuda técnica del controller.

 **Sofía**

4 DÍAS

ALTA

Validaciones y Manejo de Errores en Ejercicios de Audio (Backend)

OBJETIVO PRINCIPAL

Agregar validaciones robustas al endpoint de evaluación de audio (`POST /exercises/read-aloud`) para manejar correctamente archivos inválidos, tamaños excesivos y formatos no soportados.

CONTEXTO TÉCNICO

El `exercises.controller.ts` actual recibe el archivo sin ninguna validación (`file: any`). Si el cliente envía un archivo corrupto o un formato no soportado, el servidor puede explotar de forma no controlada. Hay que usar los `FileValidator` de NestJS (`MaxFileSizeValidator` , `FileTypeValidator`) y agregar DTOs con `class-validator` para el body.

CRITERIOS DE ACEPTACIÓN (DEFINITION OF DONE)

- ☐ El endpoint valida que el archivo sea audio (webm, ogg, mp4) y no supere 10MB.
- ☐ El body tiene un DTO con validaciones (`expectedText` , `locale` , `contentId` son requeridos).
- ☐ Los errores devuelven mensajes claros con el status HTTP correcto (400, 413, 415).
- ☐ El tipo `any` del controller está reemplazado por un tipo correcto.

 **Santiago**

4 DÍAS

ALTA

Dashboard del Alumno: Conectar Datos Reales de Progreso (Frontend)

OBJETIVO PRINCIPAL

Verificar y completar la integración del `DashboardComponent` del web-game para que muestre el progreso real del alumno (XP, monedas, sesiones jugadas) usando los endpoints del backend.

CONTEXTO TÉCNICO

La ruta `/dashboard` en `web-game` ya existe, pero hay que verificar que consuma datos reales. El backend tiene endpoints en `analytics` y `economy` . Coordinar con Felipe (Sprint 1 y 2) para que el token de Auth0 esté disponible y fluya correctamente en los HTTP calls del dashboard.

CRITERIOS DE ACEPTACIÓN (DEFINITION OF DONE)

- ☐ El dashboard muestra el saldo de monedas y XP reales del usuario logueado.
- ☐ Las métricas de progreso cognitivo vienen del backend (no de datos mockeados).
- ☐ Se implementan estados de carga (skeleton/spinner) mientras llegan los datos.
- ☐ Si el backend falla, el dashboard muestra un estado de error amigable.

 **Fernando Gómez**

4 DÍAS

ALTA

Agregar Tipos de Juego Faltantes al Game Registry (Backend + Lib)

OBJETIVO PRINCIPAL

Registrar en el `game-registry` los tipos de juego que existen en `game-types/` pero aún no están conectados al motor de evaluación del backend.

CONTEXTO TÉCNICO

En `libs/game-engine/core/src/lib/game-types/` hay 12 tipos de juego implementados (`avatar-game`, `balance-game`, `categorization-game`, etc.). Hay que auditar cuáles de ellos tienen su contraparte en el backend (endpoints en `exercises` o `evaluative`) y cuáles no. Por cada uno sin endpoint, crear el stub del servicio para que el motor pueda procesarlos.

CRITERIOS DE ACEPTACIÓN (DEFINITION OF DONE)

- ☐ Documento/comentario de qué `game-types` tienen backend y cuáles no (auditoría).
- ☐ Al menos 3 `game-types` nuevos con su endpoint básico en el backend.
- ☐ El `game-registry` tiene las entradas correspondientes actualizadas.
- ☐ El motor de evaluación no rompe al recibir un `game-type` desconocido (fallback claro).

Por favor, revisen sus tareas asignadas. Si detectan un impedimento o dependencia cruzada, repórtenlo inmediatamente en el Daily Standup. PRs específicos y documentados.